AI TECHNICAL DEBT PRIORITIZER

Development and DevOps

Services, data, interfaces, pipelines, deployment and operations

Purpose. This document describes how the AI Technical Debt Prioritizer (TDP) is built and run as software: the system architecture, the code layout, the analysis engine, the job pipeline, the API, the database, the dashboard, the CLI and GitHub integration, the MLOps workflow, testing, CI/CD, containers, deployment, security, observability and the delivery plan.

Companion documents. Document 1 explains the ML, AI and algorithms that this software runs. Document 3 lists the changes made to the original idea and why.

Conventions. Code listings are reference sketches that show structure and intent; names and versions are pinned in the real lockfiles. Performance numbers are targets, not measurements.

Contents

1. Scope and Requirements 4

1.1 Functional requirements 4

1.2 Non-functional requirements 4

2. System Architecture 4

3. Repository Layout 6

4. Technology Stack 7

5. The Analysis Engine (tdp-core) 7

5.1 Stages 7

5.2 Fast history mining 8

5.3 Content-addressed caching 9

5.4 Parallelism and limits 9

5.5 Language plug-ins 9

6. Job Processing 9

6.1 Progress streaming 10

7. Backend API (FastAPI) 10

7.1 Endpoints 10

7.2 Design rules 11

8. Data Model (PostgreSQL) 11

9. Frontend (Next.js) 12

9.1 Pages 12

9.2 Implementation notes 13

10. ML Engineering and MLOps 13

10.1 Offline pipeline 13

10.2 Reproducibility 13

10.3 Model bundle 14

10.4 Promotion gates 14

10.5 Train/serve parity and monitoring 14

11. CLI, GitHub Action and SARIF 15

11.1 CLI 15

11.2 GitHub Action 15

12. Testing Strategy 16

13. Continuous Integration and Delivery 17

14. Containers and Local Development 18

15. Deployment 20

16. Security 20

17. Observability 21

18. Delivery Plan 21

18.1 Definition of done 22

1. Scope and Requirements

1.1 Functional requirements

ID	Requirement
F1	Analyse a public Git repository from a URL (and a local path through the CLI) at a chosen branch or commit.
F2	Show progress while an analysis runs, and keep results so the same commit is never analysed twice.
F3	Rank every source file by priority, with risk probability, reasons, debt types and an estimated fix effort.
F4	Produce a budgeted, dependency-ordered fix plan for a given number of hours.
F5	Show the dependency graph, a treemap of the codebase and the debt trend across analyses.
F6	Run what-if simulations on actionable features of a file.
F7	On request, generate grounded refactoring advice for a file with an LLM.
F8	Collect agree/disagree feedback on rankings.
F9	Run in CI as a GitHub Action: comment on pull requests that touch high-priority files and emit SARIF.
F10	Provide a reproducible offline pipeline that builds the dataset, trains, evaluates and registers models.

Table 1. Functional requirements.

1.2 Non-functional requirements

Quality	Target
Performance	Cold analysis of a repository with 5,000 commits and 1,000 files in under 5 minutes on 4 vCPUs; incremental re-analysis after a few new commits in under 30 seconds.
Responsiveness	API list endpoints p95 under 300 ms; dashboard first contentful paint under 2 s on a laptop.
Reproducibility	Every score records the model version, feature-extractor version and commit SHA; training is reproducible from a pinned dataset manifest.
Safety	Analysed code is never executed; untrusted input is isolated; secrets never reach the browser or logs.
Portability	Whole stack starts with one command (Docker Compose) on Windows, macOS or Linux.
Maintainability	Typed Python and TypeScript, linting in CI, at least 80% line coverage on the core package.

Table 2. Non-functional requirements (targets).

2. System Architecture

TDP is a small set of services around one shared Python package, *tdp-core*, that contains all analysis and ML logic. The API, the workers, the CLI, the GitHub Action and the offline training pipeline all import the same package, so the features computed at training time are exactly the features computed at inference time.

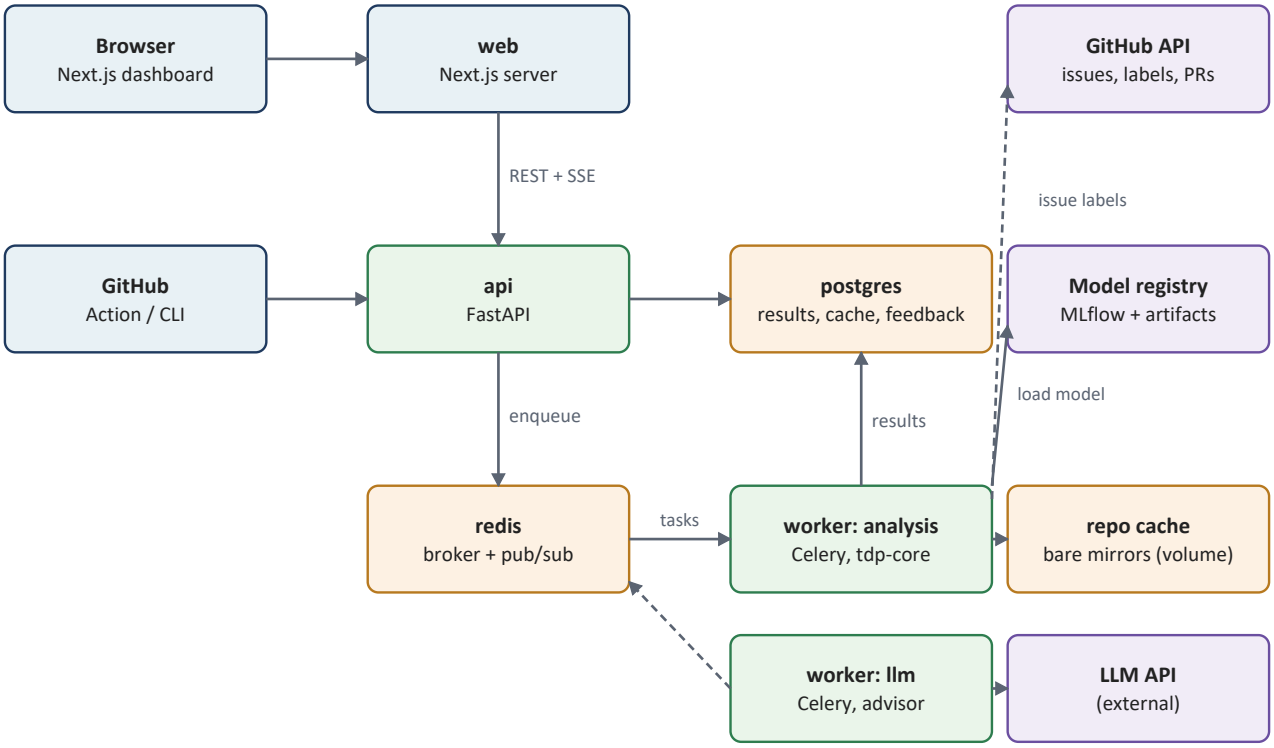


Figure 1. Runtime architecture. Green: TDP services. Amber: state. Purple: external systems and the model registry. The GitHub Action can also run tdp-core directly without the server.

Component	Responsibility	Scales by
web	Server-rendered dashboard, typed API client, no secrets	stateless replicas
api	Validation, auth, enqueueing, read endpoints, SSE progress, webhooks	stateless replicas
worker: analysis	Clone/fetch, mining, parsing, graphs, features, scoring, SHAP, planning	more processes (CPU bound)
worker: llm	Advisor calls, caching, validation; separate queue so slow LLM calls never block analyses	concurrency (I/O bound)
postgres	Source of truth: repositories, analyses, results, caches, plans, feedback, model versions	vertical, then read replica
redis	Celery broker and result backend, pub/sub for progress, rate-limit counters	single node is enough
repo cache	Bare mirrors of analysed repositories for fast incremental fetches	disk size
model registry	Versioned model bundles with metrics; workers load the bundle marked production	n/a

Table 3. Components and responsibilities.

3. Repository Layout

A single monorepo keeps the Python workspace (managed with *uv*), the Next.js app and the infrastructure together, so one pull request can change an API field, its client type and its UI.

Monorepo layout

```
tdp/
  packages/
    core/                       # tdp-core: all analysis + ML logic, no web framework imports
      tdp_core/
        mining/                 # git log parsing, mirrors, renames, bulk-commit filter
        parsing/                # tree-sitter loaders, queries per language, complexity, comments
        graph/                  # import resolution, import graph, co-change graph, centrality
        features/               # feature families S E O D I T Q, normalisation, schema
        labeling/               # bug-fix detection, SZZ, latent-defect rule (offline only)
        model/                  # bundle loading, ranker, calibrator, SHAP explainer, reasons
        planning/               # cost model, knapsack, SCC + topological scheduling, what-if
        advisor/                # LLM prompt, schema, validation
        report/                 # markdown, JSON and SARIF writers
      tests/
  services/
    api/                        # FastAPI app: routers, schemas, deps, auth, SSE
    worker/                     # Celery app: tasks wrapping tdp-core stages
  apps/
    web/                        # Next.js (App Router, TypeScript, Tailwind)
  cli/                          # `tdp` command (Typer) on top of tdp-core
  action/                       # GitHub Action (action.yml + entrypoint)
  ml/
    configs/                    # repos.yaml, ranker.yaml, labels.yaml
    pipelines/                  # tdp-ml commands: manifest, mine, features, labels, train, evaluate
    notebooks/                  # exploration only; never imported
  infra/
    docker/                     # Dockerfiles
    compose.yaml                # local and single-VM deployment
    cdk/                        # optional AWS deployment (CDK, TypeScript)
  .github/workflows/            # ci.yml, images.yml, deploy.yml, train.yml
  pyproject.toml  uv.lock  Makefile  .pre-commit-config.yaml
```

Rule that keeps the design clean

tdp-core never imports FastAPI, Celery or SQLAlchemy. It takes paths and plain data, and returns dataclasses and Arrow/Pandas tables. Services adapt it to HTTP, queues and the database. This is what lets the same code run in a worker, in the CLI, inside a GitHub Action and in the training pipeline.

4. Technology Stack

Layer	Choice	Why
Language (backend, ML)	Python 3.12, uv workspace	ML ecosystem; uv gives fast, locked, reproducible environments
Parsing	Tree-sitter (py-tree-sitter + grammar packages)	Fast, error-tolerant, many languages, same API
Git	git CLI (log, blame, fetch) + GitPython / PyDriller	CLI parsing is fastest for bulk history; libraries for SZZ and blame
Graphs	NetworkX (+ SciPy sparse for co-change)	PageRank, betweenness, SCC, topological sort built in
Data	Pandas + PyArrow (Parquet)	Columnar datasets for training; zero-copy to XGBoost
ML	XGBoost, scikit-learn, SHAP, Optuna	Ranker, calibrator and baselines; TreeSHAP; tuning
Experiment tracking	MLflow (tracking + model registry)	Runs, metrics, artifacts and model stages in one place
API	FastAPI, Pydantic v2, Uvicorn	Typed request/response models, automatic OpenAPI
Jobs	Celery + Redis	Retries, time limits, task chains, separate queues; well documented
Database	PostgreSQL 16, SQLAlchemy 2 (async), Alembic	Relational integrity + JSONB for feature vectors; migrations
Frontend	Next.js (App Router), TypeScript strict, Tailwind CSS, shadcn/ui	Server components, type safety, fast styling
Charts	Recharts (bars, histograms, treemap, sparklines)	React-native charts, simple API
Network graph	Cytoscape.js (or react-force-graph)	Recharts cannot draw node-link graphs
Data fetching	TanStack Query + openapi-typescript client	Caching, retries, types generated from the API schema
LLM	Anthropic API (Claude Sonnet 5.5 / Haiku 5.5)	Strong code reasoning; structured JSON output
CI/CD	GitHub Actions, GHCR	Same platform as the code and the Action
Containers	Docker, Docker Compose	One-command local and single-VM deployment
Observability	structlog, Prometheus, Grafana, Sentry, Flower	Logs, metrics, errors and queue visibility

Table 4. Technology choices.

5. The Analysis Engine (tdp-core)

5.1 Stages

An analysis is a fixed sequence of stages. Each stage reads the outputs of the previous ones and writes its own, which makes stages individually testable, cacheable and restartable.

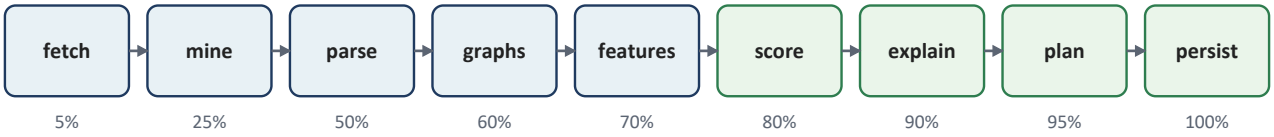


Figure 2. Analysis stages and the progress percentage reported after each one.

Stage	What it does	Cache key
fetch	Create or update a bare mirror (git clone --mirror / git fetch --prune); resolve the ref to a SHA	repository URL
mine	Parse git log once into a commits table and a file-changes table; follow renames; flag bulk commits	(repo, last mined SHA): only new commits are parsed
parse	Tree-sitter parse of each source file at the SHA: functions, complexity, comments, imports, tokens	(blob SHA, extractor version)
graphs	Resolve imports to files; build import and co-change graphs; centrality and SCCs	(analysis)
features	Assemble the feature vector for every file; percentiles; schema check	(analysis)
score	Load the production model bundle; rank; calibrate; priority bands	(analysis, model version)
explain	TreeSHAP; family grouping; reason templates; debt types	(analysis, model version)
plan	Cost model; default 40-hour plan; dependency-ordered schedule	(analysis, budget, cost config)
persist	Bulk insert results with COPY; mark the analysis succeeded; publish final event	n/a

Table 5. Stage responsibilities and cache keys.

5.2 Fast history mining

Walking commits one by one through a Python git library is slow for large histories. The miner runs a single *git log* process and streams its output:

One-pass history extraction

```
git -C <mirror> log <sha> --no-merges -M --numstat --date=unix \
    --format='%x1e%H%x1f%an%x1f%ae%x1f%at%x1f%P%x1f%s'
# %x1e (record separator) starts each commit, %x1f (unit separator) splits fields,
# then one "added<TAB>deleted<TAB>path" line per changed file (renames as old => new)
```

The parser turns the stream into two Arrow tables (*commits*, *file_changes*) that every later stage queries with Pandas. Blame for SZZ is only needed offline for labels and for the *prior latent defects* feature, and its results are cached per (fix commit, file).

5.3 Content-addressed caching

Static metrics depend only on a file's content. Git already names content by hash (the blob SHA), so static results are stored under *(blob SHA, extractor version)*. When a repository is re-analysed after ten new commits, only the handful of changed blobs are parsed again; every other file's metrics come from the cache. Changing the extractor bumps its version, which invalidates the cache automatically.

5.4 Parallelism and limits

- Parsing uses a process pool (one process per CPU) because Tree-sitter work is CPU bound and the GIL would serialise threads.
- Files above 1 MB, minified files (very long lines), generated files (headers such as "auto-generated") and vendored directories (*vendor/*, *third_party/*, *node_modules/*) are skipped and listed in the report.
- Per-file parse timeout of 5 s and a per-analysis wall-clock limit (15 minutes by default).
- History window: by default the last 3 years or 20,000 commits, whichever is smaller.

5.5 Language plug-ins

Each language is a small plug-in: a Tree-sitter grammar, query files for functions, branches, comments and imports, an import resolver and test-file naming rules. Python ships first; Java is the next plug-in.

Language plug-in interface

```
class LanguagePlugin(Protocol):
    name: str
    extensions: tuple[str, ...]
    def parse(self, source: bytes) -> FileSyntax: ...            # functions, complexity, comments, tokens
    def imports(self, syntax: FileSyntax) -> list[RawImport]: ...
    def resolve(self, imp: RawImport, index: ModuleIndex) -> str | None: ...   # repo-relative path
    def is_test_file(self, path: str) -> bool: ...
```

6. Job Processing

Mining a large repository takes minutes, far longer than an HTTP request should stay open. The API therefore only validates and enqueues; Celery workers do the work and report progress.

Celery task chain (sketch)

```
from celery import chain
from tdp_worker.app import app

common = dict(bind=True, acks_late=True, autoretry_for=(TransientGitError, OperationalError),
              retry_backoff=True, retry_jitter=True, max_retries=3)

@app.task(**common, soft_time_limit=600, time_limit=660, queue="analysis")
def mine(self, analysis_id: str) -> None:
    with stage(analysis_id, "mine", pct=25):        # updates the row and publishes a progress event
        ctx = load_context(analysis_id)
        tdp_core.mining.update(ctx.mirror, ctx.sha, ctx.store)

def start_analysis(analysis_id: str):
    return chain(fetch.si(analysis_id), mine.si(analysis_id), parse.si(analysis_id),
                 graphs.si(analysis_id), features.si(analysis_id), score.si(analysis_id),
                 explain.si(analysis_id), plan.si(analysis_id), persist.si(analysis_id)
                 ).apply_async(link_error=mark_failed.s(analysis_id))
```

- **Idempotency:** an analysis is identified by (repository, commit SHA, model version). Re-submitting the same triple returns the existing analysis instead of starting a new one, and each stage overwrites its own outputs, so

a retried stage is safe.

- **Acks late:** a task is acknowledged only after it finishes, so a worker crash puts it back on the queue.
- **Time limits:** a soft limit raises an exception the stage can clean up after; a hard limit kills the process.
- **Separate queues:** *analysis* (CPU heavy, concurrency = cores) and *llm* (I/O bound, higher concurrency), so advice requests never wait behind a large analysis.
- **Concurrency control:** a Redis lock per repository prevents two workers fetching into the same mirror at once.
- **Failure reporting:** the failing stage and a user-safe message are stored on the analysis; the stack trace goes to Sentry, not to the user.

6.1 Progress streaming

Each stage publishes *{stage, pct, message}* to the Redis channel *analysis:<id>*. The API exposes it as Server-Sent Events, which are simpler than WebSockets for one-way updates and work through most proxies. A client that connects late first receives the current state from Postgres, then live events.

7. Backend API (FastAPI)

7.1 Endpoints

Method and path	Purpose
POST /api/v1/analyses	Start (or reuse) an analysis: {repo_url, ref?, options?}. Returns 202 with the analysis id
GET /api/v1/analyses/{id}	Status, progress, commit SHA, model version, summary statistics
GET /api/v1/analyses/{id}/events	Server-Sent Events stream of progress
GET /api/v1/analyses/{id}/files	Ranked files; query: sort, order, band, debt_type, path_prefix, limit, cursor
GET /api/v1/analyses/{id}/files/{file_id}	Full detail: features, SHAP by family, reasons, history, dependents
GET /api/v1/analyses/{id}/graph	Nodes and edges for the dependency view (import + hidden-coupling edges)
GET /api/v1/analyses/{id}/treemap	Directory tree with SLOC and priority for the treemap
GET /api/v1/analyses/{id}/plan?budget_hours=40	Budgeted, ordered fix plan (computed and cached per budget)
POST /api/v1/analyses/{id}/whatif	{file_id, changes} returns new score, risk and plan effect
POST /api/v1/files/{file_id}/advice	Queue LLM advice; returns 202, then poll or stream
GET /api/v1/files/{file_id}/advice	Cached advice if ready
POST /api/v1/feedback	{file_id, verdict, comment?}
GET /api/v1/repositories/{id}/trend	Priority and debt-hour totals across past analyses
POST /webhooks/github	GitHub App webhooks (push, pull_request), verified by HMAC signature
GET /healthz, /readyz, /metrics	Liveness, readiness (DB + Redis reachable), Prometheus metrics

Table 6. REST API surface (version 1).

7.2 Design rules

- **Typed contracts:** every request and response is a Pydantic model; the OpenAPI document generated from them is committed and used to generate the TypeScript client, so a breaking change fails the web build.
- **Errors:** one JSON error shape (RFC 9457 problem details: type, title, status, detail) for every failure.
- **Pagination:** cursor-based (opaque cursor encoding the last priority and id) so pages stay stable while sorting.
- **Versioning:** the */api/v1* prefix; additive changes only within a version.
- **Async I/O:** async SQLAlchemy sessions; CPU-heavy work never runs in the API process (what-if is the exception: one model evaluation of a single vector takes milliseconds).

Starting an analysis

```
class AnalysisRequest(BaseModel):
    repo_url: HttpUrl
    ref: str | None = None                        # branch, tag or SHA; default branch if omitted
    options: AnalysisOptions = AnalysisOptions()

@router.post("/analyses", status_code=202, response_model=AnalysisCreated)
async def create_analysis(req: AnalysisRequest, db: AsyncSession = Depends(get_db),
                          user: User | None = Depends(optional_user)) -> AnalysisCreated:
    url = validate_repo_url(req.repo_url)         # https only, host allowlist, no credentials in URL
    sha = await resolve_ref(url, req.ref)         # git ls-remote: no clone needed to know the SHA
    model_version = registry.production_version()
    if (found := await analyses.find(db, url, sha, model_version)) is not None:
        return AnalysisCreated(id=found.id, status=found.status, reused=True)
    await enforce_quota(user, url)                # size and rate limits
    row = await analyses.create(db, url, sha, model_version)
    start_analysis(str(row.id))
    return AnalysisCreated(id=row.id, status="queued", reused=False)
```

8. Data Model (PostgreSQL)

PostgreSQL is required rather than optional: results must survive restarts, the same commit must never be analysed twice, trends need past analyses, and feedback must be stored for evaluation and learning.

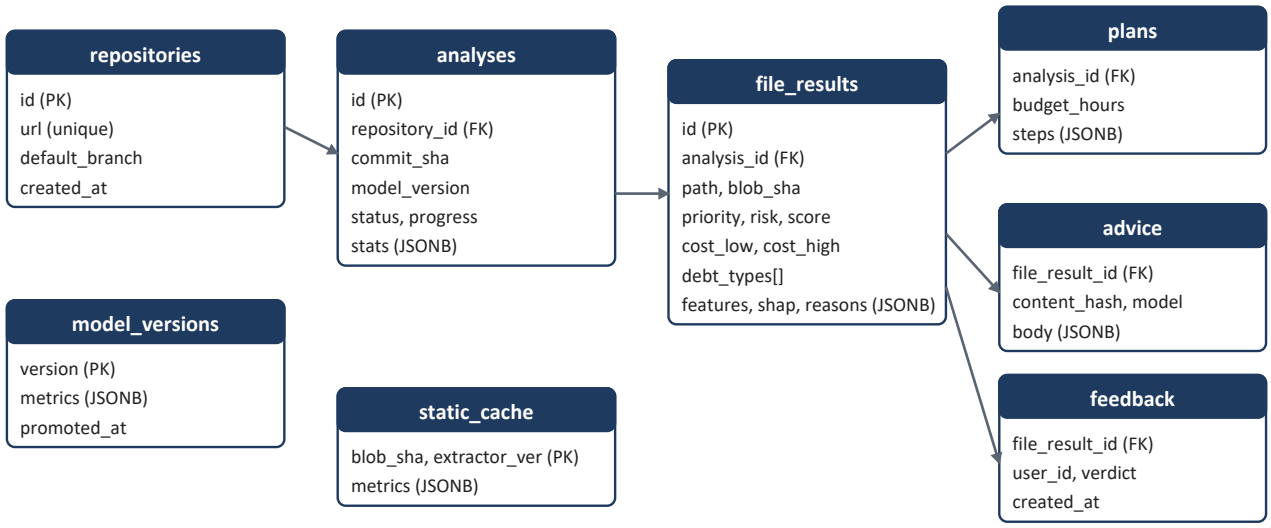


Figure 3. Core tables. Arrows point from parent to child (one-to-many).

Key DDL (excerpt)

```
CREATE TABLE analyses (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  repository_id  UUID NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  commit_sha     CHAR(40) NOT NULL,
  model_version  TEXT NOT NULL REFERENCES model_versions(version),
  status         TEXT NOT NULL CHECK (status IN ('queued','running','succeeded','failed')),
  stage          TEXT, progress SMALLINT NOT NULL DEFAULT 0, error TEXT,
  stats          JSONB,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(), finished_at TIMESTAMPTZ,
  UNIQUE (repository_id, commit_sha, model_version)          -- idempotency
);

CREATE TABLE file_results (
  id            BIGSERIAL PRIMARY KEY,
  analysis_id   UUID NOT NULL REFERENCES analyses(id) ON DELETE CASCADE,
  path          TEXT NOT NULL, blob_sha CHAR(40) NOT NULL,
  priority      SMALLINT NOT NULL, risk REAL NOT NULL, score REAL NOT NULL,
  cost_low      REAL, cost_high REAL,
  debt_types    TEXT[] NOT NULL DEFAULT '{}',
  features      JSONB NOT NULL, shap JSONB NOT NULL, reasons JSONB NOT NULL,
  UNIQUE (analysis_id, path)
);
CREATE INDEX file_results_rank  ON file_results (analysis_id, priority DESC, id);
CREATE INDEX file_results_types ON file_results USING GIN (debt_types);
```

- **JSONB for feature vectors:** the feature set evolves with model versions; JSONB avoids a migration for every new feature, while the columns the UI sorts and filters on (priority, risk, debt types) are real, indexed columns.
- **Bulk writes:** results are inserted with COPY (thousands of rows in well under a second).
- **Migrations:** Alembic, run automatically by a one-shot container before the API starts.
- **Retention:** anonymous analyses expire after 30 days (a nightly job); analyses of registered repositories are kept for trends.

9. Frontend (Next.js)

9.1 Pages

Route	Content
/	Repository URL form, examples, recent public analyses
/analyses/[id]	Live progress (SSE), then overview: KPI tiles (files analysed, Critical count, estimated hours of Critical debt), priority histogram, debt-type breakdown, top-10 table
/analyses/[id]/files	Full ranked table: sort, filter by band/debt type/path, virtualised for thousands of rows
/analyses/[id]/files/[fileId]	Priority and risk with range, reasons, SHAP family bars, commit-activity sparkline, dependents, what-if toggles, LLM advice panel, feedback buttons
/analyses/[id]/map	Treemap: area = SLOC, colour = priority; click to drill into directories
/analyses/[id]/graph	Dependency graph: node size = dependents, colour = priority, dashed edges = hidden coupling
/analyses/[id]/plan	Budget slider, ordered steps with reasons, cumulative benefit chart, export to Markdown or GitHub issues
/repositories/[id]/trend	Debt-hours and Critical count across analyses over time

Table 7. Dashboard routes.

9.2 Implementation notes

- **Rendering:** server components fetch the initial data on the server (fast first paint, no API keys in the browser); interactive parts (filters, slider, what-if) are client components using TanStack Query.
- **Typed client:** *openapi-typescript* generates types from the committed OpenAPI file; a CI step fails if the generated file is stale.
- **Charts:** Recharts for histograms, bars, the treemap and sparklines. The SHAP waterfall is a horizontal bar chart with an invisible base series. The network graph uses Cytoscape.js with a force-directed (fCoSE) layout, because Recharts has no node-link graphs.
- **Large tables:** TanStack Table with row virtualisation so 5,000 files scroll smoothly.
- **Design system:** Tailwind CSS with shadcn/ui primitives; one colour scale for priority bands that is colour-blind safe and always paired with a text label ("Critical"), never colour alone.
- **Safety:** all repository-derived text (paths, comments, advice) is rendered as text; Markdown advice goes through a sanitiser; no *dangerouslySetInnerHTML*.

10. ML Engineering and MLOps

10.1 Offline pipeline

Training is a sequence of CLI commands, each reading and writing versioned files, so any step can be re-run on its own. A scheduled GitHub Actions workflow (or a manual run on a bigger machine) executes it end to end.

Offline training commands

```
tdp-ml manifest  --config ml/configs/repos.yaml        # pins every repo to an exact commit SHA
tdp-ml mine      --manifest data/manifest.json         # mirrors + commits/file_changes parquet
tdp-ml features  --snapshot-every 6m                   # features/<repo>/<snapshot>.parquet
tdp-ml labels    --horizon 12m --gamma 0.25            # bug-fix links, SZZ, relevance grades
tdp-ml train     --config ml/configs/ranker.yaml       # Optuna search, logs runs to MLflow
tdp-ml evaluate  --protocols temporal,lopo             # metrics, baselines, ablation, plots
tdp-ml register  --run <run_id>                        # builds the bundle, stage = candidate
tdp-ml promote   --version <v>                         # only if the gates below pass
```

10.2 Reproducibility

- **Dataset manifest:** repository URLs, the commit SHA each was mined at, snapshot dates and extractor version. Its hash is recorded with every model.
- **Data files** are Parquet, stored outside git (an object store or DVC remote); the manifest and configs are in git.
- **Seeds** for XGBoost, Optuna and bootstrap are fixed and logged.
- **Environment:** the training job runs in the same worker image as production, so library versions match.

10.3 Model bundle

File	Content
ranker.json	XGBoost model in its native JSON format (portable, no pickle)
calibrator.json	isotonic regression thresholds and values as arrays (no pickle)
feature_schema.json	ordered feature names, dtypes, extractor version; checked at load time
reasons.yaml	reason templates and adjective thresholds per feature
cost_model.yaml	default remediation constants
metadata.json	version, training manifest hash, git SHA of the code, metrics, created_at

Table 8. Contents of a model bundle.

Why no pickle files

Unpickling runs code. Bundles may be downloaded from a registry or shared between machines, so every artifact is plain JSON or YAML and is loaded with a schema check. A feature-schema mismatch refuses to load instead of silently scoring with misaligned columns.

10.4 Promotion gates

A candidate becomes the production model only if all of these hold, checked automatically by *tdp-ml promote*:

- Leave-one-project-out NDCG@10 is not lower than the current production model's (within the bootstrap interval).
- It beats the churn x complexity hotspot baseline with a Wilcoxon p-value below 0.05.
- Expected calibration error is at most 0.05.
- The leakage test and the label-shuffle test pass.
- Golden-file regression: scores on three pinned repositories are produced without errors and within expected ranges.

10.5 Train/serve parity and monitoring

- Both paths call the same *tdp_core.features.build()*; a test computes features through the worker path and the training path for the same snapshot and asserts equality.
- Every analysis stores summary statistics of its feature distributions; a weekly job computes the population stability index (PSI) against the training data and flags features with PSI above 0.25 (for example, when users analyse a kind of project the model never saw).
- Feedback agreement on the top 10 is charted per model version.
- A model card (intended use, data, metrics, known limitations) is generated with each bundle.

11. CLI, GitHub Action and SARIF

11.1 CLI

Command-line interface

```
tdp analyze .                        # analyse the local checkout, print the top 20
tdp analyze https://github.com/org/repo --ref v2.3.0 --format json > report.json
tdp plan . --budget 40               # budgeted fix plan as Markdown
tdp diff --base origin/main          # only files changed since base, with their priority
tdp report . --format sarif -o tdp.sarif
```

11.2 GitHub Action

Most teams will never open a separate dashboard. The Action brings TDP into the pull-request workflow: it analyses the repository inside the CI runner (the code never leaves it), comments when a pull request touches Critical or High files, and uploads SARIF so findings appear in GitHub code scanning.

.github/workflows/tech-debt.yml in a user's repository

```
name: tech-debt
on: pull_request
permissions:
  contents: read
  pull-requests: write          # to post or update one comment
  security-events: write        # to upload SARIF
jobs:
  tdp:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }            # full history is needed for churn and ownership
      - uses: <your-org>/tdp-action@v1
        with:
          base: ${{ github.event.pull_request.base.sha }}
          comment: true
          sarif: tdp.sarif
          fail-on: none                     # advisory; teams may choose 'critical'
      - uses: github/codeql-action/upload-sarif@v3
        with: { sarif_file: tdp.sarif }
```

Pull-request comment (illustrative)

```
Technical debt check: this PR touches 2 high-priority files

| File                         | Priority      | Risk (12 mo) | Top reason                          |
|------------------------------|---------------|--------------|-------------------------------------|
| src/payments/processor.py    | 92 Critical   | 64%          | changes very often, no test imports|
| src/billing/invoice.py       | 81 High       | 41%          | 23 dependents, bug history          |

Suggestion: add or update tests for processor.py in this PR. Full report: <dashboard link>
```

- The comment is **upserted**: the Action finds its previous comment by a hidden marker and edits it, so a pull request never collects a pile of bot comments.
- The model bundle is downloaded once per runner and cached with *actions/cache*, keyed by model version.
- SARIF results map each prioritised file to a *note* or *warning* level with the reasons as the message.

12. Testing Strategy

Level	Tooling	What is tested
Unit (core)	pytest, hypothesis	complexity on fixture files with known values; import resolution; clone detection; decay weights; knapsack optimality against brute force on small random inputs; topological order properties
Git fixtures	pytest + scripted temp repos	churn, ownership, renames, bulk-commit filter, SZZ on repositories built commit by commit inside the test
ML	pytest, small synthetic dataset	training smoke test; leakage test; label-shuffle test; bundle round-trip; schema mismatch refuses to load
API	pytest + httpx AsyncClient, Postgres and Redis service containers	endpoints, validation, idempotency, pagination, error shape, SSE
Worker	Celery eager mode + integration run	stage chain on a tiny real repository; retries; failure marking
Contract	OpenAPI diff	breaking API changes are flagged in review
Frontend	Vitest + React Testing Library	components, formatting, accessibility queries
End to end	Playwright against Docker Compose	submit a repository, watch progress, open a file, change the budget
Performance	pytest-benchmark, scripted runs	analysis time on two pinned repositories, tracked over commits

Table 9. Test levels.

A git-history test built from scripted commits

```
@pytest.fixture
def tiny_repo(tmp_path):
    repo = git.Repo.init(tmp_path)
    def commit(path, text, message, days_ago, who=("Ana", "ana@example.com")):
        (tmp_path / path).parent.mkdir(parents=True, exist_ok=True)
        (tmp_path / path).write_text(text)
        repo.index.add([path])
        when = (NOW - timedelta(days=days_ago)).strftime("%Y-%m-%dT%H:%M:%S")
        return repo.index.commit(message, author=git.Actor(*who), committer=git.Actor(*who),
                                 author_date=when, commit_date=when)
    commit("pkg/a.py", "def f(x):\n    return x\n", "add f", 400)
    commit("pkg/a.py", "def f(x):\n    if x:\n        return 1\n    return x\n", "handle x", 200)
    commit("pkg/a.py", "def f(x):\n    if x > 0:\n        return 1\n    return x\n", "fix: negative x (#12)", 10)
    return repo

def test_szz_traces_fix_to_inducing_commit(tiny_repo):
    fix = tiny_repo.head.commit
    inducing = tdp_core.labeling.szz.inducing_commits(tiny_repo.working_dir, fix.hexsha)
    assert [c.summary for c in inducing] == ["handle x"]
```

13. Continuous Integration and Delivery

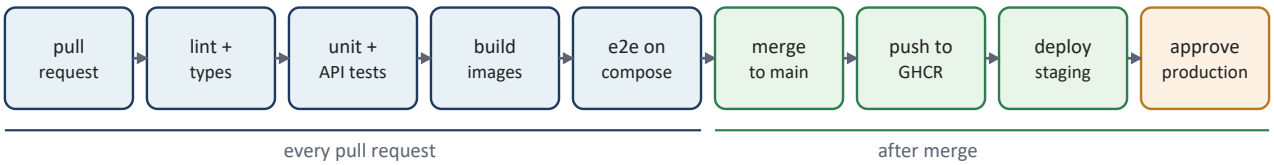


Figure 4. Delivery pipeline. Everything left of the merge runs on every pull request.

.github/workflows/ci.yml (abridged)

```
name: ci
on:
  pull_request:
  push: { branches: [main] }
permissions: { contents: read }
concurrency: { group: ci-${{ github.ref }}, cancel-in-progress: true }
jobs:
  python:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: test }
        ports: ["5432:5432"]
        options: --health-cmd pg_isready --health-interval 5s --health-retries 10
      redis: { image: "redis:7", ports: ["6379:6379"] }
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v6
      - run: uv sync --frozen
      - run: uv run ruff check . && uv run ruff format --check .
      - run: uv run mypy packages services cli
      - run: uv run pytest -n auto --cov=tdp_core --cov-fail-under=80
  web:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: apps/web } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm, cache-dependency-path: apps/web/package-lock.json }
      - run: npm ci
      - run: npm run gen:api -- --check && npm run lint && npm run typecheck
      - run: npm test -- --run && npm run build
  e2e:
    needs: [python, web]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker compose -f infra/compose.yaml up -d --build --wait
      - run: npx playwright install --with-deps chromium && npx playwright test
```

- **Pre-commit hooks** (ruff, ruff-format, prettier, end-of-file, gitleaks) catch issues before a push.
- **images.yml** builds the api, worker and web images with Buildx and layer caching, tags them with the commit SHA, generates an SBOM, scans them with Trivy and pushes them to GitHub Container Registry.
- **deploy.yml** deploys a given image tag to staging automatically and to production after a manual approval (GitHub environments with required reviewers).
- **train.yml** runs the offline pipeline on demand and opens a pull request that bumps the production model version when the promotion gates pass, so model changes are reviewed like code.

- **Dependabot** keeps Python, npm, Docker and Actions dependencies current; third-party Actions are pinned to commit SHAs.

14. Containers and Local Development

infra/docker/worker.Dockerfile

```
FROM python:3.12-slim AS base
# UV_VERSION is pinned and passed in by CI (docker build --build-arg UV_VERSION=...)
ARG UV_VERSION
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 GIT_TERMINAL_PROMPT=0
RUN apt-get update && apt-get install -y --no-install-recommends git ca-certificates \
 && rm -rf /var/lib/apt/lists/* && pip install --no-cache-dir "uv==${UV_VERSION}"
WORKDIR /app
COPY pyproject.toml uv.lock ./
COPY packages/core packages/core
COPY services/worker services/worker
RUN uv sync --frozen --no-dev --package tdp-worker
RUN useradd --create-home --uid 10001 tdp && mkdir -p /var/lib/tdp && chown tdp /var/lib/tdp
USER tdp
ENV PATH="/app/.venv/bin:$PATH"
CMD ["celery", "-A", "tdp_worker.app", "worker", "-Q", "analysis", "--concurrency", "4"]
```

infra/compose.yaml (abridged)

```
services:
  migrate:
    build: { context: .., dockerfile: infra/docker/api.Dockerfile }
    command: alembic upgrade head
    env_file: ../.env
    depends_on: { db: { condition: service_healthy } }
  api:
    build: { context: .., dockerfile: infra/docker/api.Dockerfile }
    env_file: ../.env
    ports: ["8000:8000"]
    depends_on:
      migrate: { condition: service_completed_successfully }
      redis: { condition: service_started }
  worker:
    build: { context: .., dockerfile: infra/docker/worker.Dockerfile }
    env_file: ../.env
    volumes: ["repo-cache:/var/lib/tdp/repos", "models:/var/lib/tdp/models"]
    depends_on: [api]
  worker-llm:
    build: { context: .., dockerfile: infra/docker/worker.Dockerfile }
    command: celery -A tdp_worker.app worker -Q llm --concurrency 8
    env_file: ../.env
    depends_on: [api]
  web:
    build: ../apps/web
    environment: { API_URL: "http://api:8000" }
    ports: ["3000:3000"]
    depends_on: [api]
  db:
    image: postgres:16
    environment: { POSTGRES_DB: tdp, POSTGRES_USER: tdp, POSTGRES_PASSWORD: "${POSTGRES_PASSWORD}" }
    volumes: ["pg-data:/var/lib/postgresql/data"]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U tdp"], interval: 5s, retries: 10 }
  redis:
    image: redis:7-alpine
  mlflow:
    profiles: ["ml"]                    # only started with: docker compose --profile ml up
    image: ghcr.io/mlflow/mlflow:<pinned>
    command: >
      mlflow server --host 0.0.0.0 --backend-store-uri sqlite:////mlflow/mlflow.db
      --artifacts-destination /mlflow/artifacts
    volumes: ["mlflow:/mlflow"]
    ports: ["5000:5000"]
volumes: { pg-data: {}, repo-cache: {}, models: {}, mlflow: {} }
```

- **One command:** *make up* wraps *docker compose up --build*; *make dev* runs the API and Next.js with hot reload against the containerised database and Redis.
- **Configuration** follows twelve-factor style: everything comes from environment variables, validated at startup by a Pydantic settings class; *.env.example* documents every variable; *.env* is git-ignored.
- **Next.js image** uses the standalone output mode, which copies only the files the server needs and keeps the image small.

15. Deployment

Option	How	Good for	Trade-off
Local	Docker Compose on a laptop	development, viva demo	nothing to pay; not reachable by others
Single VM (recommended for the project)	one Linux VM (4 vCPU, 8 GB), Compose, Caddy reverse proxy with automatic HTTPS, nightly pg_dump to object storage	a public demo with low cost	manual scaling; one machine to patch
Managed PaaS	web on Vercel; api and workers on a container platform; managed Postgres and Redis	less operations work	several providers and bills; workers need persistent disk for the repo cache
AWS (infrastructure as code)	AWS CDK stack: ECS Fargate services for api and workers behind an Application Load Balancer, RDS for PostgreSQL, ElastiCache for Redis, S3 for model bundles and backups, Secrets Manager, CloudWatch Logs; web on Amplify Hosting or CloudFront	a production-style reference design	highest fixed cost (database, cache, load balancer and NAT run all the time); tear down after demos

Table 10. Deployment options.

For the AWS option, every resource is defined in CDK (TypeScript) under *infra/cdk* so the environment can be created and destroyed with one command, and secrets are referenced from Secrets Manager rather than copied into task definitions. Workers need local disk for mirrors: Fargate tasks get 20 GiB of ephemeral storage by default and can be configured with more, and mirrors are a cache that can always be rebuilt.

> **Recommendation**
> Use Compose locally and on a single VM for the course demo. Keep the CDK stack as a documented, tested design that can be deployed on demand for a presentation and destroyed afterwards.

16. Security

TDP downloads and parses code chosen by strangers. That makes input handling the main security concern.

Threat	Example	Control
SSRF through the repository URL	file:///etc, http://169.254.169.254, an internal Git server	https only; host allowlist (github.com, gitlab.com, configured hosts); reject credentials in URLs; resolve DNS and block private, loopback and link-local addresses; GIT_ALLOW_PROTOCOL=https
Code execution through Git	malicious hooks, filters or submodules	never execute repository code; bare mirrors; core.hooksPath set to an empty directory; no submodule recursion; protocol.file.allow=never
Resource exhaustion	a 20 GB repository, a file with extreme nesting	size check through the hosting API before cloning; disk quota; per-file and per-analysis time limits; skip huge and generated files
Prompt injection	a comment saying "ignore previous instructions"	code is passed as quoted data; the LLM has no tools and no secrets; output must match a schema; output is shown as advisory text only
Cross-site scripting	a file path or comment containing a script tag	React escapes text; sanitised Markdown; strict Content-Security-Policy
Secret leakage	LLM key or GitHub App key in logs or the browser	secrets only in server environment or a secrets manager; log redaction; gitleaks in CI
Abuse	thousands of analyses from one client	per-IP and per-user rate limits; queue depth limit; large repositories require sign-in
Supply chain	a compromised dependency or Action	lockfiles; Dependabot; pip-audit and npm audit; Actions pinned to SHAs; Trivy image scans; SBOM

Threat	Example	Control
Private repositories	over-broad tokens	GitHub App with read-only contents and metadata permissions; short-lived installation tokens; private code is never sent to the LLM unless the owner opts in

Table 11. Threat model and controls.

- **Least privilege at runtime:** containers run as a non-root user with a read-only root filesystem except the cache and temp volumes; workers have no inbound ports.
- **Authentication** (optional for public use): GitHub OAuth via Auth.js on the web side; the API accepts a signed session token. Anonymous users can analyse small public repositories.
- **Webhooks** are verified with the HMAC signature header before any processing.

17. Observability

- **Logs:** structlog JSON lines with analysis_id, stage and repository on every line, so one analysis can be followed across the API and workers.
- **Metrics:** Prometheus endpoints on the API (request rate, latency, errors) and workers (Celery events).
- **Errors:** Sentry for API, workers and web, with release tags equal to the image SHA.
- **Queue view:** Flower for live Celery tasks, retries and failures.
- **Dashboards:** one Grafana board with the metrics below and an alert when the failure rate or queue depth stays high for 10 minutes.

Metric	Type	Why
tdp_analysis_duration_seconds{stage}	histogram	find slow stages; track the 5-minute target
tdp_analysis_total{status}	counter	success and failure rate
tdp_queue_depth{queue}	gauge	capacity planning; alerting
tdp_cache_hit_ratio{cache}	gauge	confirms incremental analysis is working
tdp_llm_tokens_total{model, kind}	counter	cost tracking
tdp_llm_validation_failures_total	counter	schema or hallucination-guard failures
tdp_feature_psi{feature}	gauge	data drift against the training set

Table 12. Key application metrics.

18. Delivery Plan

A 12-week plan with a working end-to-end slice by week 6, so integration risk is retired early. With a team, the ML track and the platform track run in parallel from week 3.

Week	ML track	Platform track	Milestone
1	repo selection, manifest, git log miner	monorepo, uv, pre-commit, CI skeleton	repo + CI green
2	Tree-sitter Python plug-in, static features, clone detection	Compose with Postgres and Redis	features for one repo
3	import and co-change graphs, ownership and evolution features	FastAPI skeleton, Alembic, Celery chain	
4	bug-fix detection, SZZ, labels, dataset build	analysis endpoints, SSE progress	full dataset built

Week	ML track	Platform track	Milestone
5	baselines, temporal + LOPO evaluation harness	Next.js shell, typed client, overview page	
6	LambdaMART ranker, tuning	file list and detail pages	end-to-end slice works
7	calibration, SHAP, reason templates, ablation	treemap and dependency graph	
8	cost model, knapsack, scheduler, debt types, SATD	plan page, what-if	fix plan in UI
9	model bundle, MLflow, promotion gates	incremental analysis, caching, metrics	
10	LLM advisor and evaluation rubric	advice panel, feedback	AI layer complete
11	sensitivity analysis, PROMISE sanity check	CLI, GitHub Action, SARIF, security hardening	Action published
12	final experiments and figures	deployment, load test, docs	report + demo

Table 13. Twelve-week plan.

18.1 Definition of done

- *make up* starts the full stack from a clean clone, and the demo repository analysis completes.
- CI is green: lint, types, unit, API, frontend and end-to-end tests; core coverage at least 80%.
- The production model passed every promotion gate, and its metrics and model card are in the registry.
- Every number in the report can be regenerated with the documented *tdp-ml* commands.
- The security controls in Section 16 are implemented and each has at least one test.